

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250314>

Outline : Training Deep Neural Networks (code [samples](#))

- Definitions / Motivation
- Vanishing/Exploding Gradients Problems
- Reusing Pretrained Layers
- Faster Optimizers
- Avoiding Overfitting Through Regularization
- Summary and Practical Guidelines

—

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapter 11**

Definition / Motivation

What if a complex problem of detecting hundreds of types of object in high-resolution images? → Much deeper DNN (>> 10 layers with hundreds of neurons and 100K connections).

This gives several challenges: i) vanishing gradients problem (or exploding gradients problem) in DNN: lower layers very hard to train; ii) not enough training data for such a large network / too costly to label; iii) training may be extremely slow; iv) model with millions of parameters would severely risk overfitting the training set, especially if there are not enough training instances, or they are too noisy.

Faster Optimizers

Faster Optimizers

- Training a very large deep neural network can be painfully slow.
- So far seen four ways to speed up training (and reach a better solution):
 - applying a good initialization strategy for the connection weights,
 - using a good activation function,
 - using Batch Normalization, and
 - reusing parts of a pretrained network (possibly built on an auxiliary task or using unsupervised learning).
- Another huge speed boost comes from using a faster optimizer than the regular Gradient Descent optimizer.
 - Momentum optimization, Nesterov Accelerated Gradient, AdaGrad, RMSProp, and Adam and Nadam optimization.

Momentum Optimization (1)

Gradient Descent simply updates the weights θ by directly subtracting the gradient of the cost function $J(\theta)$ with regards to the weights ($\nabla_{\theta} J(\theta)$) multiplied by the learning rate η . The equation is: $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$. It does not care about what the earlier gradients were.

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

Momentum Optimization (2)

Analogy: a bowling ball rolling down a gentle slope on a smooth surface: it will start out slowly, but it will **quickly pick up momentum until it eventually reaches terminal velocity** (if there is some friction or air resistance).

Momentum optimization cares a about **what previous gradients were**: at each iteration, it subtracts the local gradient from the momentum vector $\mathbf{m}$ (multiplied by the learning rate η), and it updates the weights by simply adding this momentum vector.

Hyperparameter β , called the momentum, which must be set between 0 (high friction) and 1 (no friction). β typically is 0.9.

Equation 11-4. Momentum algorithm

1. $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta + \mathbf{m}$

Momentum Optimization (3)

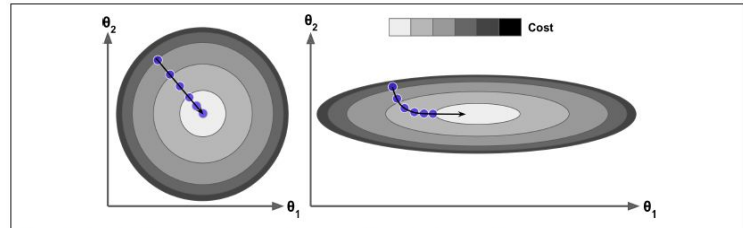


Figure 4-7. Gradient Descent with and without feature scaling

If the gradient remains constant, the terminal velocity (i.e., the **maximum size of the weight updates**) is equal to that gradient multiplied by the learning rate η multiplied by $\frac{1}{1-\beta}$ (ignoring the sign). For example, if $\beta = 0.9$, then the terminal velocity is equal to 10 times the gradient times the learning rate, so Momentum optimization ends up going 10 times faster than Gradient Descent! This allows Momentum optimization to escape from plateaus much faster than Gradient Descent.

Equation 11-4. Momentum algorithm

1. $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta + \mathbf{m}$

Momentum Optimization (4)

- Due to the momentum, the optimizer **may overshoot** a bit, then come back, overshoot again, and oscillate like this many times before stabilizing at the minimum.
- This is one of the reasons why it is good to have a bit of friction in the system: it gets rid of these oscillations and thus speeds up convergence.

```
optimizer = keras.optimizers.SGD(lr=0.001, momentum=0.9)
```

Nesterov Accelerated Gradient (1)

- A variant of momentum optimization, almost always faster than “standard” Momentum optimization.
- To measure the gradient of the cost function not at the local position but slightly ahead in the direction of the momentum.
- The only difference from standard Momentum optimization is that the gradient is measured at $\theta + \beta \mathbf{m}$ rather than at θ .
-

Equation 11-5. Nesterov Accelerated Gradient algorithm

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$$

$$2. \quad \theta \leftarrow \theta + \mathbf{m}$$

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$$

$$2. \quad \theta \leftarrow \theta + \mathbf{m}$$

Nesterov Accelerated Gradient (2)

That small tweak works because in general the momentum vector will be pointing in the right direction (i.e., toward the optimum), so it will be slightly more accurate to use the gradient measured a bit farther in that direction rather than using the gradient at the original position.

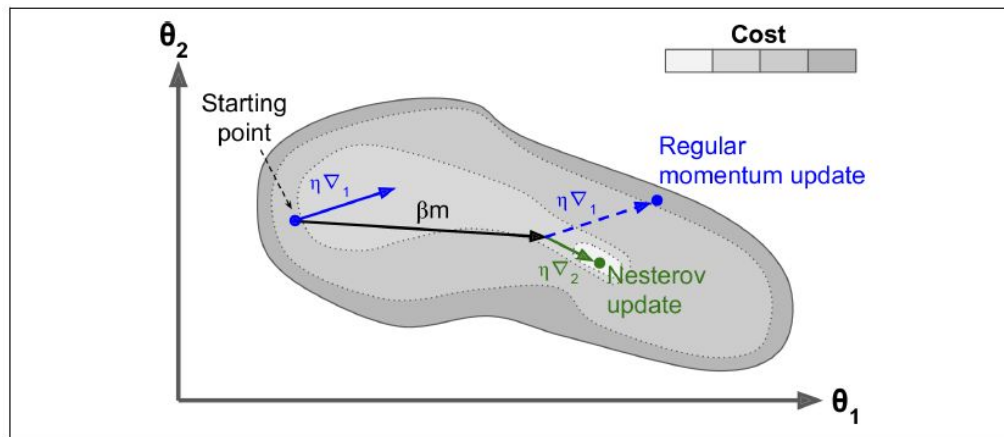


Figure 11-6. Regular versus Nesterov Momentum optimization

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$$

$$2. \quad \theta \leftarrow \theta + \mathbf{m}$$

Nesterov Accelerated Gradient (3)

```
optimizer = keras.optimizers.SGD(lr=0.001, momentum=0.9, nesterov=True)
```

- ∇_1 represents the gradient of the cost function measured at the starting point θ , and ∇_2 represents the gradient at the point located at $\theta + \beta \mathbf{m}$. The Nesterov update ends up slightly closer to the optimum. After a while, these small improvements add up and NAG ends up being significantly faster than regular Momentum optimization.
- When the momentum pushes the weights across a valley, ∇_1 continues to push further across the valley, while ∇_2 pushes back toward the bottom of the valley.

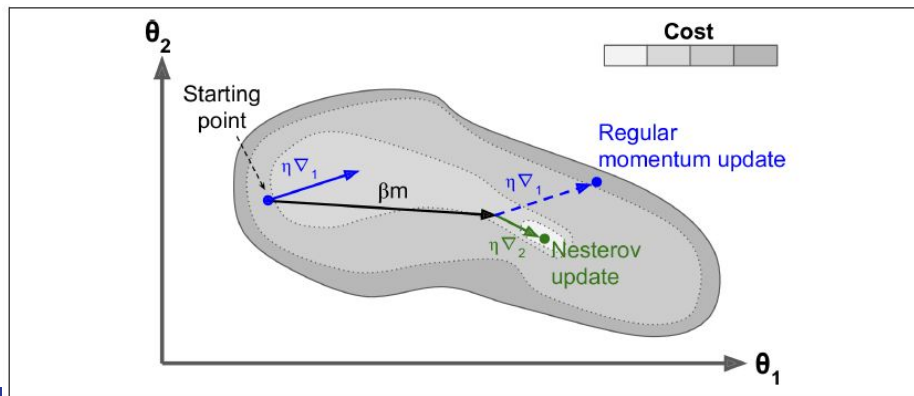


Figure 11-6. Regular versus Nesterov Momentum optimization

AdaGrad (1)

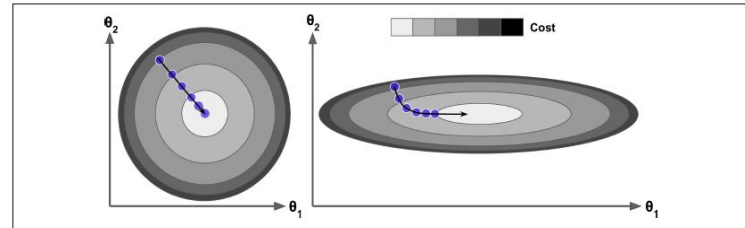


Figure 4-7. Gradient Descent with and without feature scaling

Consider the elongated bowl problem: Gradient Descent starts by quickly going down the steepest slope, then slowly goes down the bottom of the valley. It would be nice if the algorithm could detect this early on and correct its direction to point a bit more toward the global optimum.

The [AdaGrad algorithm](#) achieves this by scaling down the gradient vector along the steepest dimensions:

Equation 11-6. AdaGrad algorithm

1. $\mathbf{s} \leftarrow \mathbf{s} + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{\mathbf{s} + \epsilon}$

AdaGrad (2)

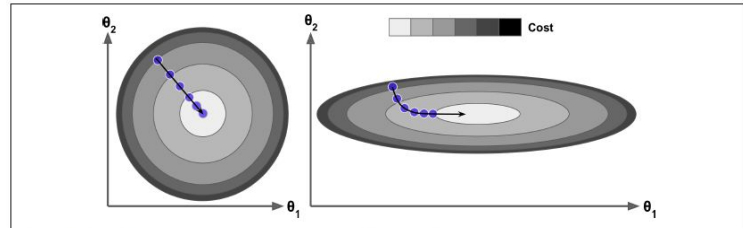


Figure 4-7. Gradient Descent with and without feature scaling

- The first step accumulates the square of the gradients into the vector $\mathbf{s}$.
- $\otimes$ symbol represents the element-wise multiplication.
- Vectorized form is equivalent to computing $s_i \leftarrow s_i + (\partial J(\boldsymbol{\theta}) / \partial \theta_i)^2$ for each element s_i of the vector $\mathbf{s}$; so, each s_i accumulates the squares of the partial derivative of the cost function with regards to parameter θ_i . Thus, if the cost function is steep along the i^{th} dimension, then s_i will get larger and larger at each iteration.

Equation 11-6. AdaGrad algorithm

1. $\mathbf{s} \leftarrow \mathbf{s} + \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
2. $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \oslash \sqrt{\mathbf{s} + \epsilon}$

AdaGrad (3)

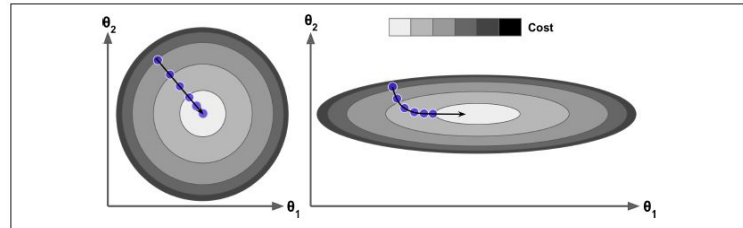


Figure 4-7. Gradient Descent with and without feature scaling

- The second step is almost identical to Gradient Descent, but with one big difference: the gradient vector is scaled down by a factor of $\sqrt{s + \epsilon}$.
- $\otimes$ represents the element-wise division
- ϵ is a smoothing term to avoid division by zero, typically set to 10^{-10} .
- This vectorized form is equivalent to computing $\theta_i \leftarrow \theta_i - \eta \partial J(\theta) / \partial \theta_i / \sqrt{s + \epsilon}$ for all parameters θ_i (simultaneously).

Equation 11-6. AdaGrad algorithm

1. $s \leftarrow s + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{s + \epsilon}$

AdaGrad (4)

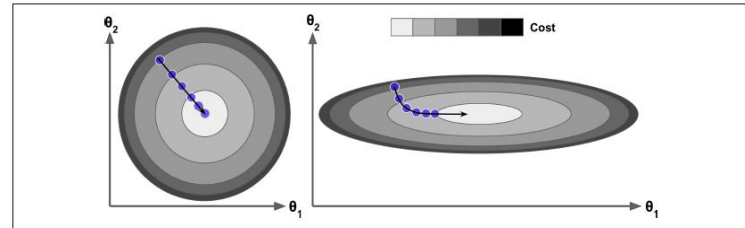


Figure 4-7. Gradient Descent with and without feature scaling

- The algorithm decays the learning rate, but it does so faster for steep dimensions than for dimensions with gentler slopes. This is called an *adaptive learning rate*.

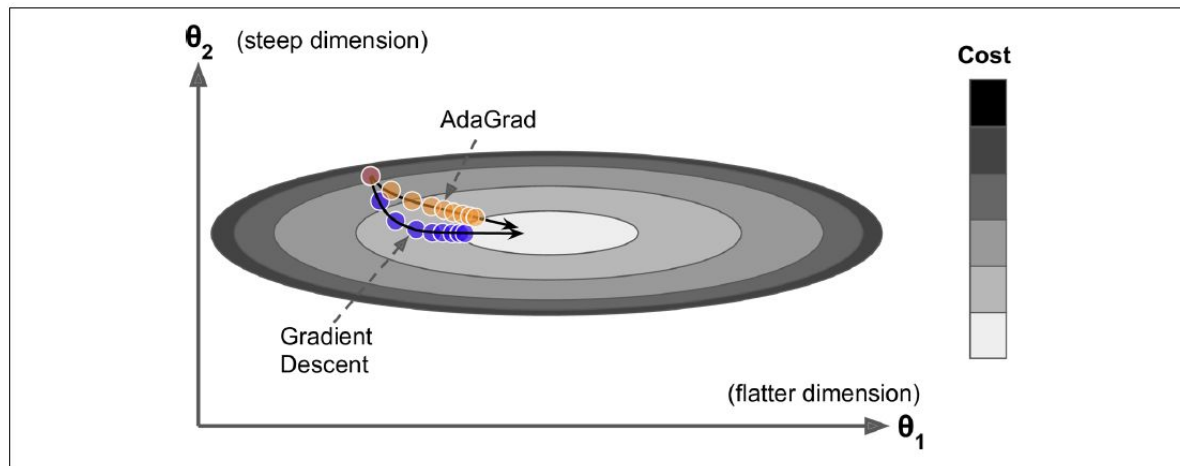


Figure 11-7. AdaGrad versus Gradient Descent

AdaGrad (5)

- AdaGrad often performs well for simple quadratic problems, but unfortunately it often stops too early when training neural networks.
- The **learning rate gets scaled down so much that the algorithm ends up stopping entirely before reaching the global optimum**. So even though Keras has an Adagrad optimizer, you should not use it to train deep neural networks (it may be efficient for simpler tasks such as Linear Regression, though).
- Understanding Adagrad is helpful to grasp the other adaptive learning rate optimizers.

RMSProp

AdaGrad slows down a bit too fast and ends up never converging to the global optimum, the *RMSProp* algorithm fixes this by accumulating only the gradients from the most recent iterations (as opposed to all the gradients since the beginning of training). The decay rate β is typically set to 0.9.

Equation 11-7. RMSProp algorithm

1. $\mathbf{s} \leftarrow \beta \mathbf{s} + (1 - \beta) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
2. $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \oslash \sqrt{\mathbf{s} + \epsilon}$

```
optimizer = keras.optimizers.RMSprop(lr=0.001, rho=0.9)
```

Adam and Nadam Optimization (1)

- Adam (*adaptive moment estimation*) combines the ideas of Momentum optimization and RMSProp.
- Just like Momentum optimization it keeps track of an exponentially decaying average of past gradients.
- Just like RMSProp it keeps track of an exponentially decaying average of past squared gradients

Equation 11-8. Adam algorithm

1. $\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla_{\theta} J(\theta)$
2. $\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
3. $\widehat{\mathbf{m}} \leftarrow \frac{\mathbf{m}}{1 - \beta_1^t}$
4. $\widehat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^t}$
5. $\theta \leftarrow \theta + \eta \widehat{\mathbf{m}} \oslash \sqrt{\widehat{\mathbf{s}} + \epsilon}$

t represents the iteration number (starting at 1).

Adam and Nadam Optimization (2)

- In steps 1, 2, and 5, Adam is similar to both Momentum optimization and RMSProp.
- The only difference is that step 1 computes an exponentially decaying average rather than an exponentially decaying sum, but these are actually equivalent except for a constant factor (the decaying average is just $1 - \beta_1$ times the decaying sum).
- Steps 3 and 4 are somewhat of a technical detail: since $\mathbf{m}$ and $\mathbf{s}$ are initialized at 0, they will be biased toward 0 at the beginning of training, so these two steps will help boost $\mathbf{m}$ and $\mathbf{s}$ at the beginning of training.
- Typically: $\beta_1 = 0.9$, the scaling decay hyperparameter $\beta_2 = 0.999$. The smoothing term $\epsilon = 10^{-7}$.

```
optimizer = keras.optimizers.Adam(lr=0.001, beta_1=0.9, beta_2=0.999)
```

Adam and Nadam Optimization (3)

- Nadam optimization: it is simply Adam optimization plus the Nesterov trick, so it will often converge slightly faster than Adam. In the report, many different optimizers are compared on various tasks → Nadam generally outperforms Adam, but is sometimes outperformed by RMSProp.

Summary

- All the optimization techniques discussed so far only rely on the first-order partial derivatives (Jacobians).
- There are amazing algorithms based on the second-order partial derivatives (the Hessians, which are the partial derivatives of the Jacobians).
- Unfortunately, these algorithms are very hard to apply to deep neural networks because there are n^2 Hessians per output (where n is the number of parameters), as opposed to just n Jacobians per output.
- Since DNNs typically have tens of thousands of parameters, the second-order optimization algorithms often do not even fit in memory, and even when they do, computing the Hessians is just too slow.

Learning Rate Scheduling (1)

- Finding a good learning rate:
 - If set way too high, training may actually diverge.
 - If set too low, training will eventually converge to the optimum, but it will take a very long time.
 - If set slightly too high, it will make progress very quickly at first, but it will end up dancing around the optimum, never really settling down.
 - If there is a limited computing budget, you may have to interrupt training before it has converged properly, yielding a suboptimal solution.

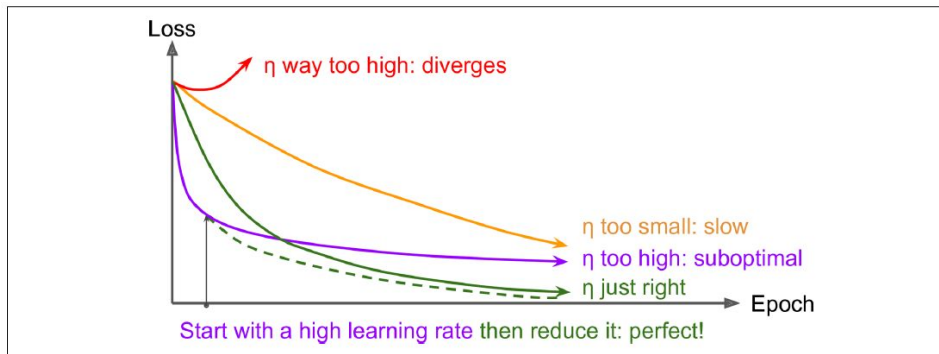


Figure 11-8. Learning curves for various learning rates η

Learning Rate Scheduling (2)

As we discussed in previous [L13](#), one approach is to start with a large learning rate, and divide it by 3 until the training algorithm stops diverging. It will not be too far from the optimal learning rate, which will learn quickly and converge to good solution.

However, there are better ways than a constant learning rate: *Learning Schedules*.

Learning Rate Scheduling (3)

- Power scheduling `optimizer = keras.optimizers.SGD(lr=0.01, decay=1e-4)`
 - Set the learning rate to a function of the iteration number t : $\eta(t) = \eta_0 / (1 + t/s)^c$. The initial learning rate η_0 , the power c (typically set to 1) and the steps s are hyperparameters. The learning rate drops at each step, and after s steps it is down to $\eta_0 / 2$. After s more steps, it is down to $\eta_0 / 3$. Then down to $\eta_0 / 4$, then $\eta_0 / 5$, and so on. As you can see, this schedule first drops quickly, then more and more slowly. Of course, this requires tuning η_0 , s (and possibly c).
- Exponential scheduling
 - Set the learning rate to: $\eta(t) = \eta_0 0.1^{t/s}$. The learning rate will gradually drop by a factor of 10 every s steps. While power scheduling reduces the learning rate more and more slowly, exponential scheduling keeps slashing it by a factor of 10 every s steps.

Learning Rate Scheduling (4)

- Piecewise constant scheduling
 - Use a constant learning rate for a number of epochs (e.g., $\eta_0 = 0.1$ for 5 epochs), then a smaller learning rate for another number of epochs (e.g., $\eta_1 = 0.001$ for 50 epochs), and so on. Although this solution can work very well, it requires fiddling around to figure out the right sequence of learning rates, and how long to use each of them.
- Performance scheduling
 - Measure the validation error every N steps (just like for early stopping) and reduce the learning rate by a factor of λ when the error stops dropping.

```
def piecewise_constant_fn(epoch):
    if epoch < 5:
        return 0.01
    elif epoch < 15:
        return 0.005
    else:
        return 0.001
```

Learning Rate Scheduling (5)

```
def exponential_decay_fn(epoch):  
    return 0.01 * 0.1**(epoch / 20)  
  
def exponential_decay(lr0, s):  
    def exponential_decay_fn(epoch):  
        return lr0 * 0.1**(epoch / s)  
    return exponential_decay_fn  
  
lr_scheduler = keras.callbacks.LearningRateScheduler(exponential_decay_fn)  
history = model.fit(X_train_scaled, y_train, [...], callbacks=[lr_scheduler])
```

Avoiding Overfitting Through Regularization

Avoiding Overfitting Through Regularization (1)

- Deep neural networks typically have tens of thousands of parameters, sometimes even millions.
- With so many parameters, the network has an incredible amount of freedom and can fit a huge variety of complex datasets.
- This great flexibility also means that it is prone to overfitting the training set.
- So, we need regularization.
- Previous examples: early stopping or Batch Normalization for vanishing/exploding gradients problems.

ℓ_1 and ℓ_2 Regularization (1)

- ℓ_1 and ℓ_2 regularization to constrain a neural network's connection weights (but typically not its biases).
- The `l2()` function returns a regularizer that will be called to compute the regularization loss, at each step during training. This regularization loss is then added to the final loss.
- `keras.regularizers.l1()` for ℓ_1 regularization.
- `keras.regularizers.l1_l2()` for both ℓ_1 and ℓ_2 .
-

```
layer = keras.layers.Dense(100, activation="elu",  
                             kernel_initializer="he_normal",  
                             kernel_regularizer=keras.regularizers.l2(0.01))
```

ℓ_1 and ℓ_2 Regularization (2)

- Python's **functools.partial()** function: to create a thin wrapper for any callable

```
from functools import partial
```

```
RegularizedDense = partial(keras.layers.Dense,  
                           activation="elu",  
                           kernel_initializer="he_normal",  
                           kernel_regularizer=keras.regularizers.l2(0.01))
```

```
model = keras.models.Sequential([  
    keras.layers.Flatten(input_shape=[28, 28]),  
    RegularizedDense(300),  
    RegularizedDense(100),  
    RegularizedDense(10, activation="softmax",  
                    kernel_initializer="glorot_uniform")  
])
```

Dropout (1)

- Dropout is one of the most popular regularization techniques for deep neural networks. Proposed in 2012 and further detailed in 2014.
- Proven to be highly successful: even the state-of-the-art neural networks got a 1–2% accuracy boost simply by adding dropout. This may not sound like a lot, but when a model already has 95% accuracy, getting a 2% accuracy boost means dropping the error rate by almost 40% (going from 5% error to roughly 3%).

Dropout (2)

At every training step, every neuron (including the input neurons, but always excluding the output neurons) has a probability p of being temporarily “dropped out,” meaning it will be entirely ignored during this training step, but it may be active during the next step. The hyperparameter p is called the dropout rate, and it is typically set to 50%. After training, neurons don’t get dropped anymore.

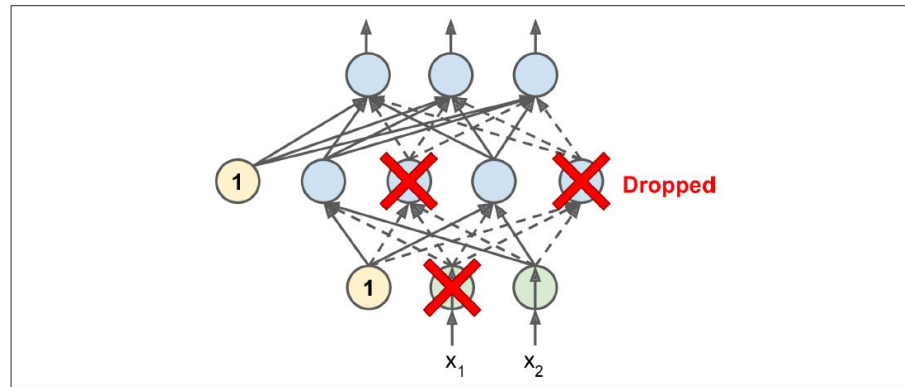


Figure 11-9. Dropout regularization

Dropout (3)

```
model = keras.models.Sequential([  
    keras.layers.Flatten(input_shape=[28, 28]),  
    keras.layers.Dropout(rate=0.2),  
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),  
    keras.layers.Dropout(rate=0.2),  
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),  
    keras.layers.Dropout(rate=0.2),  
    keras.layers.Dense(10, activation="softmax")  
])
```

Monte-Carlo (MC) Dropout

Introduced in 2016.

- MC Dropout, which can boost the performance of any trained dropout model, without having to retrain it or even modify it at all.
- Provides a much better measure of the model's uncertainty.
- Simple to implement. The full implementation of MC Dropout, boosting the dropout model that was trained earlier, without retraining it:

```
with keras.backend.learning_phase_scope(1): # force training mode = dropout on
    y_probas = np.stack([model.predict(X_test_scaled)
                        for sample in range(100)])
y_proba = y_probas.mean(axis=0)
```

Summary and Practical Guidelines

Summary and Practical Guidelines (1)

The following work fine in most cases, without requiring much hyperparameter tuning.

Table 11-2. Default DNN configuration

Hyperparameter	Default value
Kernel initializer:	LeCun initialization
Activation function:	SELU
Normalization:	None (self-normalization)
Regularization:	Early stopping
Optimizer:	Nadam
Learning rate schedule:	Performance scheduling

Hyperparameter	Default value
Kernel initializer:	LeCun initialization
Activation function:	SELU
Normalization:	None (self-normalization)
Regularization:	Early stopping
Optimizer:	Nadam
Learning rate schedule:	Performance scheduling

Summary and Practical Guidelines (2)

- If the model self-normalizes:
 - If it overfits the training set, then you should add alpha dropout (and always use early stopping as well). Do not use other regularization methods, or else they would break self-normalization.
- If the model cannot self-normalize (e.g., it is a recurrent net or it contains skip connections):
 - You can try using ELU (or another activation function) instead of SELU, it may perform better. Make sure to change the initialization method accordingly (e.g., He init for ELU or ReLU).
 - If it is a deep network, you should use Batch Normalization after every hidden layer. If it overfits the training set, you can also try using max-norm or ℓ_2 regularization.

Hyperparameter	Default value
Kernel initializer:	LeCun initialization
Activation function:	SELU
Normalization:	None (self-normalization)
Regularization:	Early stopping
Optimizer:	Nadam
Learning rate schedule:	Performance scheduling

Summary and Practical Guidelines (3)

- If you need a sparse model, you can use ℓ_1 regularization (and optionally zero out the tiny weights after training). If you need an even sparser model, you can try using FTRL instead of Nadam optimization, along with ℓ_1 regularization. In any case, this will break self-normalization, so you will need to switch to BN if your model is deep.
- If you need a low-latency model (one that performs lightning-fast predictions), you may need to use less layers, avoid Batch Normalization, and possibly replace the SELU activation function with the leaky ReLU. Having a sparse model will also help. You may also want to reduce the float precision from 32-bits to 16-bit (or even 8-bits).

Summary and Practical Guidelines (4)

Hyperparameter	Default value
Kernel initializer:	LeCun initialization
Activation function:	SELU
Normalization:	None (self-normalization)
Regularization:	Early stopping
Optimizer:	Nadam
Learning rate schedule:	Performance scheduling

- If you are building a risk-sensitive application, or inference latency is not very important in your application, you can use MC Dropout to boost performance and get more reliable probability estimates, along with uncertainty estimates.